BRAHMO Clinical AI Demo — End-to-End Working Guide

Prototype v0.1.0 · Streamlit Community Cloud + Supabase · Engineering demo only. Not for clinical use, not a medical device, no real patient data.

1. What the app does

The app is a thin slice of the BRAHMO architecture. A doctor-facing Streamlit UI sends input down one of two paths, and every step that matters is written to a durable audit log in Supabase.

- **Clinical reference path:** question -> lexical retrieval over the fictional `knowledge/` files -> evidence sufficiency and applicability gate -> grounded answer made of excerpts and citations (no LLM) -> doctor review -> audit event.
- **Medication review path:** product lines -> drug identity graph (synthetic catalogue) -> ingredient expansion -> deterministic safety rail (eight-state contract) -> doctor review -> audit event.
- **Audit:** `brahmo/audit.py` writes to Supabase when configured, always keeps an in-memory copy, and also writes a local SQLite file on a developer machine. Only Supabase is durable on Streamlit Community Cloud.

2. Project layout

- `app.py` — Streamlit UI (five tabs, sidebar, diagnostics buttons)
- `brahmo/audit.py` — audit events and the source-version registry (Supabase client)
- `brahmo/orchestrator.py` — runs both pipelines and writes the pipeline audit events
- `brahmo/safety.py`, `drug_graph.py`, `retrieval.py`, `contracts.py` — safety rules, drug catalogue, retrieval, data contracts
- `knowledge/` — fictional demo corpus used by the reference path
- `supabase/schema.sql` — the two tables and their security settings
- `tests/test_safety.py` — tests, run with `pytest -q`
- `requirements.txt` — `streamlit`, `supabase`, `pytest`
- `.streamlit/config.toml` — headless server settings

3. Supabase setup (once)

1. Create a project at supabase.com.
2. Open **SQL Editor**, paste the contents of `supabase/schema.sql` and run it. This creates `audit_events` and `source_versions`, turns on row-level security, and revokes all access from `anon` and `authenticated`.
3. Open **Project Settings -> API Keys** and copy the **secret** key (starts with `sb_secret_`). On the legacy tab this is the `service_role` key.
4. Do not use the **publishable** key (starts with `sb_publishable_`) or the `anon` key. The schema blocks them by design, and using one gives `permission denied for table audit_events` (code 42501).
5. Never paste the secret key into GitHub, chat or any file in the repo. It bypasses row-level security.

To confirm the schema is in place, run:

```
select table_name from information_schema.tables
where table_schema = 'public'
  and table_name in ('audit_events','source_versions');
```

Both names should come back.

4. Deploy to Streamlit Community Cloud

1. Push the repo to GitHub on the `main` branch. The root must contain `app.py`, `requirements.txt`, `brahmo/`, `knowledge/` and `.streamlit/config.toml`.
2. In share.streamlit.io choose **Create app**, pick the repo and `main`, set the main file to `app.py`, and deploy.
3. Open **Manage app -> Settings -> Secrets** and add these two lines at the top level, with straight double quotes and no `[section]` header:

```
SUPABASE_URL = "https://YOUR_PROJECT_REF.supabase.co"
SUPABASE_SERVICE_ROLE_KEY = "sb_secret_..."
```

4. **Reboot the app** from Manage app. Saving secrets alone does not restart the Python process, and an older deployed `audit.py` keeps the client it built first.
5. Every later `git push` to `main` redeploys automatically.

5. Local development

```
python -m venv .venv
source .venv/bin/activate      # Windows: .venv\Scripts\activate
pip install -r requirements.txt
pytest -q
streamlit run app.py
```

To log to Supabase locally, set the same names as environment variables before running:

```
export SUPABASE_URL="https://YOUR_PROJECT_REF.supabase.co"
export SUPABASE_SERVICE_ROLE_KEY="sb_secret_..."
```

The `.env` file is already in `.gitignore`. Without these variables the app still runs and logs to memory and a local SQLite file at `data/audit_demo.sqlite`.

6. Using the app, tab by tab

1. **Sidebar:** set a synthetic Tenant ID and Encounter ref, and the population filter. Under Persistence it should say **Supabase connected (durable audit)**.
2. **Tab 1, End-to-end pipeline:** choose Clinical reference question or Medication review. Run the pipeline and read the result. Tick the doctor-confirmation box to record a confirmation. The box is unticked automatically when a new result is produced.
3. **Tab 2, Medication review:** try `Demo Product A`, `Demo Product B`, `Demo Product C`. Product C is ambiguous and stays unresolved, so the check returns a partial result. Duplicate ingredient tokens trigger a `HIT` on the demo rule.
4. **Tab 3, Clinical reference:** ask about the assessment section, versioning or eligibility. Setting the population filter to pediatric makes the app abstain on adult-only sources.
5. **Tab 4, Eight states:** shows the safety contract and the rule coverage manifest.
6. **Tab 5, Audit & sources:** shows recent audit events and the source-version registry. Registering a source writes to `source_versions` and also logs an audit event.

7. What gets logged

Action in the app	event_type	event_status
Run medication pipeline	`medication_review_pipeline`	e.g. `PARTIAL`
Run reference pipeline	`clinical_reference_pipeline`	`GROUNDED` or `ABSTAINED`
Tick the confirmation box	`doctor_confirmation_demo`	`CONFIRMED` (one row per tick)
Register a source version	`source_version_registered`	`OK`

The `source_versions` table is unique on `(source_name, version_label)`. Registering the same pair again updates that row (including status and hash) instead of adding a new one. Use a different name or version label to add rows.

8. Verify in Supabase

Latest events:

```
select id, event_type, event_status, created_at
from public.audit_events
order by created_at desc
limit 10;
```

Counts per event type:

```
select event_type, count(*) from public.audit_events group by event_type order by 2 desc;
```

Registered sources:

```
select source_name, version_label, status, created_at
from public.source_versions order by created_at desc;
```

9. Built-in diagnostics

The sidebar has two buttons that are useful when something breaks.

- **Test Supabase write** reports whether the URL and key are present, what kind of key it is, the `supabase` package version, and the exact error if an insert or the source-versions upsert fails. It writes a `diagnostic_ping` row.
- **Test audit logger path** sends a `logger_path_test` event through the same function the pipelines use and shows whether the insert failed.
- The **Registration failed** message in tab 5 now shows the same diagnostic JSON.

Delete the test rows afterwards:

```
delete from public.audit_events where event_type in ('diagnostic_ping','logger_path_test');
delete from public.source_versions where source_name = 'diagnostic_ping';
```

10. Troubleshooting

Symptom	Cause	Fix
`permission denied for table ...` (42501)	Publishable or anon key in the secrets	Use the `sb_secret_` / `service_role` key
`could not find the table` (PGRST205)	Schema not run	Run `supabase/schema.sql` in the SQL Editor
Sidebar says Supabase not configured	Secrets missing, misspelled, or inside a `[section]`	Use the exact names above at the top level, then reboot

Symptom	Cause	Fix
Test button works, pipeline rows missing	Logger still holds a client built with the old key	Reboot the app; deploy the `audit.py` that rebuilds the client when the key changes
`ImportError: cannot import name ...`	`app.py` and `brahmo/audit.py` are out of sync on GitHub	Push both files, `audit.py` inside `brahmo/`, then reboot
Several confirmation rows per click	Old version wrote the event on every rerun	Use the current `app.py` (event written once per tick)
Only one row in `source_versions`	Upsert on the same name and version	Register a different name or version
Sidebar says connected but nothing is stored	The check only proves the client exists	Run **Test Supabase write**

11. Before showing it to anyone

- Remove the test rows (section 9) and, if you like, the two diagnostic buttons from the sidebar.
- Check that no secret key appears anywhere in the repo history. If one ever did, rotate it in Project Settings -> API Keys.
- Use only synthetic data: `Demo Product A/B/C`, `TEST_INGREDIENT_*` tokens, pseudonymous encounter refs. Never enter real patient details.
- Keep the safety notice visible. Outputs must not guide diagnosis, prescribing or care decisions.

12. Known limits and sensible next steps

- Not implemented: a real drug master, interaction/allergy/pregnancy rules, a regulatory event feed, a production RAG/LLM, authentication, tenant isolation, encryption for PHI and clinical approvals.
- The audit table can currently be updated or deleted by anyone with the secret key. Before any pilot, add an append-only guard (revoke update/delete or add a trigger that blocks them).
- A **Register all knowledge files** button that hashes each file in `knowledge/` would keep the source registry in step with the real corpus.
- Before any clinical pilot, the README lists the required clinical review, legal and regulatory advice, source-rights checks, access controls and held-out evaluation.